University of Nottingham Ningbo China

Department of Computer Science

Final Year Project Proposal

Multi-Objective Optimization for Large-Scale Neural Solver Deployment for Combinatorial Optimization

---

Name: Yuxuan LI
Student ID: 20616316
Email: scyyl36@nottingham.edu.cn
Programme of Study: BSc Computer Science and Artificial Intelligence
Supervisor: Ruibin BAI

9 October 2026

1 Background and Motivation

Combinatorial optimization problems such as the travelling salesman problem (TSP) and the capacitated vehicle routing problem (CVRP) underpin logistics, scheduling and network design. Neural combinatorial optimization (NCO) now solves them competitively with classical heuristics, through constructive models [8, 9], heatmap-and-search methods [7, 13], learned improvement operators [6, 12] and decomposition [4, 16]. These are almost all research results. What a trained solver costs to *run*, as opposed to how good its tours are, has received little systematic attention. This project addresses that gap. Its subject is the deployment-time inference efficiency of NCO solvers: where the cost of a fixed, trained solver goes when it is executed repeatedly in production, and how much of that cost can be removed without retraining the model.

Reporting conventions obscure the question. A typical paper quotes a single pair of numbers — solution quality and runtime — measured on one accelerator, with runtime treated as an undifferentiated wall clock. That conceals[1] how inference time decomposes into arithmetic, kernel launch, host–device synchronisation and Python dispatch[2]; how much of it is recoverable by the techniques a deployment engineer actually controls; and[3] how the answers change with the number of instances served, the degree of concurrency and the length of the run. The magnitude is not academic: on TSP-1000, POMO with 8-fold augmentation is reported at a 40.57% optimality gap and 8.5 minutes, against 3.168% and 1.6 minutes for a greedy heatmap-plus-search method [11]. A difference that large decides whether a method is deployable at all, yet it becomes visible only when quality and latency are reported together across scales.

Where inference cost is addressed at all, the treatment is algorithmic rather than systemic. StruDiCO reports a 3.5× speedup and 70% memory reduction by restructuring the denoising process [14], and Fast T2T reaches order-of-magnitude gains by replacing iterative training-to-testing solving with a consistency objective [10]. Both redesign the model; neither asks what a *fixed* solver costs to run, or how much of that cost falls to compilation, batching, reduced precision and memory management — the levers available without touching the model. One level further out, the Amortized Efficiency Threshold (AET) asks how many instances must be served before a neural solver's training energy is repaid by its per-instance inference advantage, reporting a crossover near 1.58×10^5 instances at n=20 [1]. That result is validated at a single scale on a single hardware configuration, so whether the threshold rises, falls or inverts under deployment-scale conditions is unknown.

My reading is that this gap is structural rather than a matter of further tuning. In the NCO literature "large-scale" almost always denotes large instance size; for deployment the binding constraint is usually how many instances arrive, how many are served concurrently, and how long the service runs. A survey of analytics and machine learning in vehicle routing likewise identifies the distance between what learning-based methods demonstrate in research and what operators can run in practice as a recognised open problem [2]. I will therefore characterise neural solver cost along that deployment axis, holding the solver fixed rather than proposing a new one, so that the measurements describe a system rather than a contribution. The platform is held fixed as well: every measurement is taken on a single NVIDIA GPU workstation, which isolates solver configuration from platform variation and makes the measured quality–latency trade-off a property of the solver rather than of the machine that ran it. Generality is tested instead across solver families — constructive, heatmap-based and improvement methods — which asks the same deployment question of three different kinds of model.

实验环境和实际部署之间的差距。

2 Aim and Objectives

Aim. The aim of this project is to characterise where inference cost is spent in neural combinatorial optimization solvers at deployment scale, to quantify how much of that cost is recoverable through system-level optimisation, and to derive a decision rule that identifies the cost-optimal solver configuration for a given workload and solution-quality constraint.

Objectives. The following objectives specify the individual aspects that must be delivered to achieve the aim.

- **O1. Establish a reproducible measurement environment.** Pin the reference codebase to one revision and hold it read-only; generate instance sets once, with fixed seeds, and reuse them across every run; and repeat a fixed calibration workload at the start of every measurement session, so that drift in clocks, thermals or driver state is detected rather than mistaken for an effect of the code.
- **O2. Characterise where inference time is spent.** Profile a reference workload to attribute time to arithmetic, kernel launch, host synchronisation and Python dispatch, and determine how that attribution migrates as instance size, batch size and decoding budget increase.
- **O3. Quantify how much of the inference cost is recoverable.** Measure the effect of compilation, batching and reduced precision against an eager baseline, one intervention at a time, and then restructure the autoregressive decode loop, whose per-step host synchronisation to test a termination condition lies outside any compiled subgraph. Establish whether the total headroom grows or shrinks with instance size.
- **O4. Formulate deployment as a multi-objective problem.** Trace the Pareto front over solution quality and latency under a quality constraint, and derive the decision rule the aim specifies.
- **O5. Test the result beyond the base setting.** Repeat the key measurements on a second problem family (TSP to CVRP) and across solver families; compute the Amortized Efficiency Threshold against a strong heuristic baseline, charging the full training cost, and test how the threshold varies with instance size, batch size and model family. Identify which conclusions transfer and which reverse.

3 Project Plan

The project runs for six months, October 2026 to March 2027. Tasks map to the objectives in §2, and the plan front-loads the environment and profiling work that everything else depends on, so that a usable result exists well before the deadline rather than only at the end.

- **T1 — Environment, baselines and calibration** (October; O1). Pin RL4CO [3] to a single revision; establish LKH-3 [5] and PyVRP [15] as reference solvers; fix the calibration workload that every later session repeats.
- **T2 — Characterisation and profiling** (November to mid December; O2). Profile Attention Model on TSP-100 with CPU and GPU profilers; sweep instance size and batch size; produce the hotspot-migration result.
- **T3 — Optimisation I** (mid December to January; O3). Run compilation, batching and precision ablations against the eager baseline, one intervention at a time so that effects remain attributable.

- **T4 — Optimisation II** (January to mid February; O3). Replace the variable-length decode loop with a fixed-length loop; measure synchronisation count, device idle time and end-to-end latency.
- **T5 — Multi-objective evaluation** (mid to end February; O4). ε-constraint sweep, Pareto front, hypervolume comparison and the resulting decision rule.
- **T6 — Deployment cost** (late February to mid March; O5). Compute the AET against the heuristic baseline, including full training cost and sensitivity over instance size, batch size and hardware.
- **T7 — Generalisation** (late February to late March; O5). Repeat the key measurements on CVRP and on two further solver families — heatmap-based and improvement methods against the autoregressive baseline; identify where conclusions reverse.
- **T8 — Writing and reproducibility package** (February to March, drafting throughout). Assemble the report and a package that regenerates every table and figure from raw measurements by script.

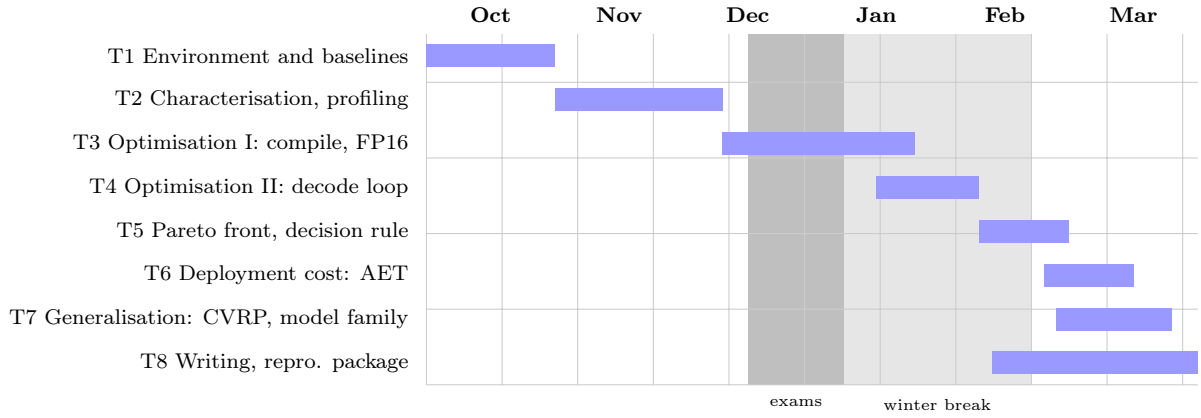


Figure 1: Six-month project plan. Shaded bands mark periods of reduced availability: examinations (mid December to early January, darker) and the winter break that follows (to late February, lighter).

Scheduling constraints.

- **Reduced availability.** Two windows are shaded in Figure 1: examinations, mid December to early January, and the winter break that follows, to late February.
- **Tasks that overlap them.** T3, T4, part of T5 and the drafting strand of T8 are the least machine-intensive of the project — analysis, ablation and writing, which tolerate interruption. The uninterrupted profiling sweeps and thermal measurements of T2, T6 and T7 fall outside both windows.
- **Contingencies.** Energy measurement requires a Linux environment for per-process attribution; failing that, the AET half of O5 is deferred and the rest of the project is unaffected. If the larger batch and instance sizes of T7 exhaust GPU memory, the affected configurations are recorded as a reported boundary rather than a failed run — the memory ceiling is itself a deployment result.

References

[1] Anonymous. An amortized efficiency threshold for comparing neural and heuristic solvers, 2026. Author list to be completed.

[2] R. Bai, X. Chen, Z.-L. Chen, T. Cui, S. Gong, W. He, X. Jiang, H. Jin, J. Jin, G. Kendall, et al. Analytics and machine learning in vehicle routing research. *International Journal of Production Research*, 61(1):4–30, 2023.

[3] F. Berto, C. Hua, J. Park, L. Luttmann, Y. Ma, F. Bu, J. Wang, H. Ye, M. Kim, S. Choi, et al. RL4CO: an extensive reinforcement learning for combinatorial optimization benchmark, 2023.

[4] L. Duan, J. Wang, Z. Yang, Y. Shi, and D.-C. Zhan. H-TSP: Hierarchically solving the large-scale travelling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[5] K. Helsgaun. An extension of the Lin–Kernighan–Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. Technical report, Roskilde University, 2017.

[6] A. Hottung, Y.-D. Kwon, and K. Tierney. Efficient active search for combinatorial optimization problems. In *International Conference on Learning Representations (ICLR)*, 2022.

[7] C. K. Joshi, T. Laurent, and X. Bresson. An efficient graph convolutional network technique for the travelling salesman problem, 2019.

[8] W. Kool, H. van Hoof, and M. Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.

[9] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[10] Y. Li, J. Guo, R. Wang, H. Zha, and J. Yan. Fast T2T: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[11] F. Luo, X. Lin, F. Liu, Q. Zhang, and Z. Wang. Instance-conditioned adaptation for large-scale generalization of neural combinatorial optimization, 2024.

[12] Y. Ma, J. Li, Z. Cao, W. Song, L. Zhang, Z. Chen, and J. Tang. Learning to iteratively solve routing problems with dual-aspect collaborative transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[13] Z. Sun and Y. Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization, 2023.

[14] Y. Wang, Y. Li, J. Yan, and Y. Chang. StruDiCO: Structured denoising diffusion with gradient-free inference-stage boosting for memory and time efficient combinatorial optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[15] N. A. Wouda and L. Lan. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 2024.

[16] H. Ye, J. Wang, Z. Liang, Y. Liang, Z. Cao, Y. Li, and F. Li. GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time, 2023.